

Description of the Microservice Design Patterns and Antipatterns Examined in the Dissertation

This appendix first describes the patterns detected by the proposed pattern-mining approach introduced in Section 3-1-1 and the 13 microservice patterns supported by the proposed benchmark presented in Section 3-2. It then introduces the antipatterns supported by the proposed antipattern-mining approach presented in Section 3-1-2. The pattern descriptions are based on a review of the relevant literature (Valdivia, Lora-González, Limón, Cortes-Verdin, & Ocharán-Hernández, 2020), together with other sources, including Richardson’s book (Richardson, 2018) and authoritative online resources (Richardson, 2025; Microsoft, 2025; Bhojwani, 2025).

A Microservice Design Patterns Examined in the Dissertation

This section briefly describes the microservice patterns supported by the proposed pattern-detection approach presented in Section 3-1-1 and by the proposed benchmark introduced in Section 3-2. The patterns were selected according to two criteria: (1) they represent well-established and widely used architectural solutions in microservice-based systems (Ramchand, 2021; Valdivia et al., 2020; Sebastian et al., 2023; Richardson, 2018, 2025; Microsoft, 2025; Bhojwani, 2025); and (2) they can be detected through static analysis of system-level and microservice-level artifacts, including dependencies, annotations, imports, and configuration files.

1. **Asynchronous Messaging:** This pattern enables non-blocking communication among microservices by sending and receiving messages through message brokers or queues such as RabbitMQ and Kafka.
2. **Circuit Breaker:** The Circuit Breaker pattern prevents cascading failures in distributed systems. It monitors service calls and activates fallback or delay mechanisms when a service becomes unresponsive. Widely used frameworks for implementing this pattern include Hystrix and Resilience4j.
3. **Event Sourcing:** Event Sourcing stores every change to an application’s state as an immutable sequence of events rather than retaining only the current state.
4. **Application Programming Interface Gateway (API Gateway):** The API Gateway centralizes client-request management, routing, authentication, and load distribution across multiple microservices.
5. **Service Discovery:** The Service Discovery pattern supports the dynamic registration and discovery of service instances in a distributed system, thereby facilitating scalability and fault tolerance.
6. **Monitoring:** The Monitoring pattern provides comprehensive visibility into the runtime behavior, performance, and health of microservices.
7. **Cache-Aside:** The Cache-Aside pattern improves system performance by loading data into the cache only when it is requested and keeping the cache consistent with the underlying database.
8. **Command Query Responsibility Segregation (CQRS):** This pattern separates the system’s read and write responsibilities into distinct components, thereby improving scalability and performance.
9. **Distributed Tracing:** Distributed Tracing makes it possible to track requests across microservice boundaries to analyze latency, dependencies, and bottlenecks.
10. **Automatic Configuration:** This pattern automatically configures microservices according to their runtime environment, reducing manual configuration and improving maintainability.
11. **Saga:** The Saga pattern manages long-running distributed transactions across multiple microservices by decomposing them into a sequence of local transactions coordinated through events or an orchestrator.
12. **Service Instance per Execution Environment:** This pattern isolates each microservice within its own execution environment, improving deployment independence, scalability, and fault isolation. It can be detected through Dockerfiles or execution-environment descriptors in the microservice source code.
13. **Pipes and Filters:** This pattern organizes processing as a sequence of independent stages (filters) connected by pipes, making data transformations modular and reusable. Detection is based on pipeline-oriented parallel-processing frameworks, imports, and annotations that identify filter or pipeline components.
14. **Leader Election:** This pattern enables a group of microservice instances to dynamically elect a single coordinator (leader) that coordinates shared tasks and ensures consistency among distributed components.
15. **Proxy:** The Proxy pattern introduces an intermediary service that routes, filters, or transforms requests between clients and downstream microservices. It centralizes cross-cutting concerns such as authentication, rate limiting, and request routing while reducing coupling between clients and backend services.
16. **Aggregator:** The Aggregator pattern combines the results of multiple microservices and presents them as a single response. In a microservice architecture, a client request may require data or functionality distributed across several services. An integration layer forwards requests to the relevant services, collects and processes their results, and returns the consolidated output to the client.
17. **Priority Queue:** The Priority Queue pattern manages and processes tasks or messages according to priority. Each message or task is assigned a priority level, and the queue is designed to process higher-priority items first.
18. **Sidecar:** The Sidecar pattern allows microservices to use external or non-Java services while retaining Java-based architectural features such as an API Gateway and Service Discovery. A sidecar container or process runs alongside the primary microservice and supplies the capabilities required for integration with the Spring ecosystem or the microservice infrastructure.
19. **Ambassador:** The Ambassador pattern integrates external or non-Java services with the Spring Boot ecosystem or other microservices. It introduces a local intermediary or proxy that provides facilities such as an API Gateway, Service Discovery, and communication configuration for an external service. Its primary objective is to allow external services to benefit from microservice-architecture capabilities without internal modification while simplifying their deployment and management in distributed environments.

B Microservice Antipatterns Examined in the Dissertation

This section describes the 15 microservice antipatterns supported by the proposed antipattern-mining approach. These antipatterns were selected through a comprehensive review of the literature aimed at identifying and classifying microservice-related antipatterns (Taibi, Lenarduzzi, & Pahl, 2020; Cerny, Abdelfattah, Al Maruf, Janes, & Taibi, 2023). The selection criteria were as follows: (1) the antipatterns are commonly observed in microservice-based systems and have been investigated in multiple studies; and (2) they can be detected using internal system features and microservice-level properties obtainable through static analysis and system parsing. Antipatterns that require knowledge of runtime interactions among microservices or external behavioral dependencies were therefore excluded from the scope of this research.

1. **Wrong Cuts:** This antipattern occurs when microservices are designed and decomposed according to technical layers—such as presentation, business logic, or data access—rather than business capabilities. Such decomposition weakens internal cohesion, increases inter-service coupling, and conflicts with the principles of Domain-Driven Design (DDD) and microservice autonomy.
2. **God Service:** In this antipattern, a single microservice implements several business functions. Concentrating excessive responsibilities and processes in one microservice reduces cohesion, increases internal dependencies, and undermines team autonomy and system scalability.
3. **Nano Service:** A Nano Service performs only a very narrow and simple task and, when considered independently, often provides no meaningful business value. It increases inter-service communication complexity and introduces substantial overhead for deployment, monitoring, and infrastructure-configuration management.
4. **Shared Libraries:** In this antipattern, several microservices depend on shared libraries that typically contain common logic, utility functions, or shared data models. Although such reuse may initially appear beneficial, it violates microservice independence and creates tight coupling among services that should remain autonomous.
5. **Hard-Coded Endpoints:** This antipattern refers to explicitly embedding endpoint details—such as URLs, internal port addresses, or port numbers—in source code or configuration files. Such rigid configuration reduces flexibility, disrupts automated deployment processes such as CI/CD, and increases the likelihood of runtime misconfiguration.
6. **Manual Configuration:** This antipattern arises when microservices are configured manually and in a decentralized manner. It reduces maintainability, limits scalability, and hinders automation, all of which are essential to the effective operation of microservice-oriented systems.
7. **No Continuous Integration/Continuous Delivery:** This antipattern denotes the absence, incomplete implementation, or manual execution of automated processes for integrating code developed by different teams and deploying it to testing or production environments. Consequently, software delivery becomes slower, more error-prone, and less reliable, adversely affecting overall system quality and agility.
8. **No API Gateway:** In this antipattern, no dedicated API Gateway manages user requests and directs them to the appropriate microservices. Users therefore communicate directly with microservices, increasing complexity, security risks, traffic-management challenges, and undesirable coupling between users and services.
9. **No Timeout:** This antipattern occurs when microservice consumers repeatedly attempt to connect to an unresponsive service because timeout settings are absent or service availability is unknown. The result is unnecessary retries, degraded performance, and inefficient resource utilization.
10. **Multiple Service Instances per Host:** In this antipattern, multiple instances of identical or different microservices are deployed manually or without appropriate coordination on the same physical or virtual host. This creates challenges in service isolation, resource allocation, scalability, and fault tolerance.
11. **Shared Persistence:** This antipattern occurs when multiple microservices use a shared database or schema rather than maintaining independent persistence. The practice creates tight coupling, restricts deployment autonomy, and increases the risk of data inconsistency and concurrency problems.
12. **No Versioning:** In this antipattern, changes to a microservice's application programming interface are introduced without an appropriate versioning mechanism. Such changes can break existing clients and disrupt dependent microservices.
13. **No Health Check:** This antipattern denotes the absence of dedicated endpoints or mechanisms for reporting the operational state of microservices. Without health checks, the system cannot identify unstable or failed services and may continue routing requests to them, causing degraded service quality or system outages.
14. **Local Logging:** In the Local Logging antipattern, each microservice stores its runtime logs locally without centralized collection or analysis. This reduces system observability and makes debugging and monitoring more difficult.
15. **Insufficient Monitoring:** This antipattern arises when development or operations teams fail to implement adequate mechanisms for observing microservice behavior. Consequently, the system lacks real-time visibility into performance and health, making effective detection, analysis, and response to problems more difficult.

References

- Bhojwani, R. (2025). *Design patterns for microservices*. Retrieved from <https://dzone.com/articles/design-patterns-for-microservices> (Accessed 31 December 2025)
- Cerny, T., Abdelfattah, A. S., Al Maruf, A., Janes, A., & Taibi, D. (2023). Catalog and detection techniques of microservice anti-patterns and bad smells: A tertiary study. *Journal of Systems and Software*, 206, 111829. doi: 10.1016/j.jss.2023.111829
- Microsoft. (2025). *Design patterns for microservices*. Retrieved from <https://learn.microsoft.com/en-us/azure/architecture/microservices/design/patterns> (Accessed 31 December 2025)
- Ramchand, M. (2021). *A systematic mapping of microservice patterns* (Bachelor's thesis, University of Groningen). Retrieved from <https://fse.studenttheses.ub.rug.nl/25671/>
- Richardson, C. (2018). *Microservices patterns: with examples in Java*. Simon and Schuster.
- Richardson, C. (2025). *Microservice architecture pattern*. Retrieved from <https://microservices.io/patterns/microservices.html> (Accessed 31 December 2025)
- Sebastian, S., et al. (2023). A systematic review on the identification and classification of patterns in microservices. In *2023 4th international conference for emerging technology (incet)* (pp. 1–8). doi: 10.1109/incet57972.2023.10170375
- Taibi, D., Lenarduzzi, V., & Pahl, C. (2020). Microservices anti-patterns: A taxonomy. In *Microservices: Science and engineering* (pp. 111–128). Springer. doi: 10.1007/978-3-030-31646-4_5
- Valdivia, J. A., Lora-González, A., Limón, X., Cortes-Verdin, K., & Ocharán-Hernández, J. O. (2020). Patterns related to microservice architecture: a multivocal literature review. *Programming and Computer Software*, 46(8), 594–608. doi: 10.1134/s0361768820080253